# CyberAware: Click. Think. Protect.

**Applied Research Project — Final Report**

**Course:** CSIS4495  Section 002
**Term:** W26

**Student / Team lead:** Janvi Arora
**Student ID:** 300383801

## Table of Contents


1. Introduction
    - 1.1 Domain, Background, and Context
    - 1.2 Problem Framing
    - 1.3 Literature and Related Work
    - 1.4 Knowledge Gaps Addressed
    - 1.5 Hypotheses and Assumptions
2. Summary of the Research Project
3. Changes to the Proposal
4. Project Completion Timeline and Responsibilities
    - 4.1 Timeline
    - 4.2 Responsibilities
    - 4.3 Project Management Approach
5. Implemented Features
    - 5.1 Feature 1: Authentication and Security
    - 5.2 Feature 2: Structured Learning Modules
    - 5.3 Feature 3: Mission Board and UI Design
    - 5.4 Feature 4: Progress Tracking and Database Integration
6. Evaluation Techniques
    - 6.1 Build Verification
    - 6.2 Functional Testing
    - 6.3 Usability Evaluation
    - 6.4 Limitations
    - 6.5 Informal User Feedback
7. Reflections and Discussion
8. AI Use
9. Work Date and Hours Logs
10. Concluding Remarks
11. References
12. Appendix A: Installation Guide
13. Appendix B: User Guide
14. Appendix C: AI Prompt History

# 1. Introduction

## 1.1 Domain, background, and context

Cybersecurity awareness training addresses a persistent gap between **technical controls** (firewalls, endpoint protection, identity systems) and **human decision-making**. Attackers routinely bypass strong technology by targeting people through **phishing**, **credential reuse**, **social engineering**, and **malicious web content**. Organizations therefore invest in awareness programs; however, many programs still rely on static slide decks or annual compliance videos that produce **low engagement** and **weak retention** (SANS Security Awareness, 2023; NIST SP 800-50 outlines a lifecycle for awareness and training programs).

The **web application** domain is a natural delivery channel for awareness content: it supports **interactivity**, **immediate feedback**, **personalized progress**, and **remote access**. Modern **backend-as-a-service** platforms (e.g. Supabase, Firebase) allow student-scale projects to implement **real authentication** and **database-enforced security** (Row-Level Security) without maintaining a custom server, which aligns with course learning outcomes in secure application design.

**CyberAware** sits in this space: a **single-page application (SPA)** that presents a **structured learning path** across six cybersecurity topics, combined with **gamification** (experience points, levels, achievements) and **persistent per-user progress** stored in a relational database protected by **Row-Level Security (RLS)**.

## 1.2 Problem framing

This research addresses the following questions:

1. **How can a lightweight web platform combine pedagogy (learn → apply → assess) with measurable engagement signals (XP, badges, sequential unlocks) without sacrificing basic security hygiene?**

2. **What implementation and data-model choices best align a React frontend with Supabase Auth and Postgres so that users can only access their own progress?**

3. **What usability and functional risks appear when integrating third-party auth (session refresh, missing configuration, empty database rows), and how can the UI remain stable under those conditions?**

These questions matter because **misconfigured client apps** (e.g. missing environment variables) and **ambiguous database schemas** (e.g. `.single()` when zero rows exist) are common sources of **406/404 errors**, blank screens, and **loss of user trust** during demos and pilots.

## 1.3 Literature and related work

Security awareness research emphasizes **repeated exposure**, **interactive exercises**, and **contextual scenarios** rather than one-off lectures (Kumaraguru et al., 2007 on phishing education; downstream work on embedded training and simulated phishing). **Gamification** in education can increase motivation when goals, feedback, and autonomy are well aligned (Deterding et al., 2011), though poorly tuned extrinsic rewards may distract from learning objectives. **OAuth-style** and **email/password** flows with **multi-factor authentication** are standard mitigations for account takeover (NIST SP 800-63B). For student projects, **RLS** provides a concrete mechanism to enforce **least privilege** at the data layer (PostgreSQL policies tied to `auth.uid()`).

## 1.4 Knowledge gaps addressed

Many tutorials demonstrate **login** or **CRUD** in isolation; fewer show an **end-to-end training journey** with **sequential content**, **scenario branching**, **quizzes with explanations**, and **aggregated progress JSON** synchronized to a remote database. This project contributes a **coherent vertical slice**: UX, content design, schema, and defensive coding patterns (e.g. avoiding brittle `.single()` assumptions; using `maybeSingle` or `limit(1)` where appropriate).

## 1.5 Hypotheses, assumptions, and expected benefits

**Hypotheses (practical, engineering-oriented):**

- **H1:** Learners complete more steps when the path is **visually structured** (mission board, module cards) and **progress is visible** (XP, levels).

- **H2: Scenario + quiz** pairs improve self-reported confidence compared to reading alone (informal evaluation; not a randomized controlled trial).

- **H3:** A **single-row-per-user** progress document in JSON reduces join complexity for a six-module MVP compared to a fully normalized per-question schema.

**Assumptions:** Users have a modern browser; email/password auth is acceptable for the pilot; Supabase project is administered correctly (redirect URLs, RLS enabled). **Benefits:** Reusable codebase for portfolio and future extension (admin analytics, more modules, localization).

---

# 2. Summary of the research project (final form)

CyberAware is a **React + Vite** application deployed as static assets, backed by **Supabase Authentication** and **PostgreSQL**. Authenticated users access a **dashboard**, a **module**

**board** with **flippable cards** and **sequential unlocking**, six **module experiences**, an **achievements** page, and **profile** editing.

Each module follows a consistent **ModuleDetail** pattern:

- **Part 1 — Learn:** Key points and a **comic strip** component.
- **Part 2 — Scenario:** Narrative prompt (e.g. suspicious email) with **selectable responses** and **feedback**.
- **Part 3 — Quiz:** Multiple-choice items with **post-submit explanations**; pass threshold is at least **half** correct; XP is granted on pass.
- **Part 4 — Real-world threat example:** Short narrative linking the topic to realistic risk (e.g. phishing module includes **look-alike URL** discussion and a **calendar-invite / fake subscription cancellation** scam story).

**Gamification:** Points accumulate into **levels** (Beginner through Expert in the UI logic). **BadgeContext** awards badges such as first module, halfway, all modules, and high XP, persisting earned badge IDs in **`user_badges`**.

**Security posture:** Database tables **`profiles`**, **`user_progress`**, and **`user_badges`** are protected with **RLS** policies restricting reads/writes to the authenticated user. The client uses the **anon** key only; no service role key ships to the browser. Environment variables **`VITE_SUPABASE_URL`** and **`VITE_SUPABASE_ANON_KEY`** configure the client; missing configuration is detected so the shell does not fail with an opaque runtime error.

Repository layout highlights: `Implementation/frontend_app/` contains the runnable app; `supabase/schema.sql` and `supabase/SUPABASE_SETUP.md` document database setup; `ReportsAndDocuments/` and `DocumentsAndReports/` hold submission artifacts.

---

## 3. Changes to the proposal

This section outlines the key changes made from the initial proposal submitted earlier in the course to the final implementation of the CyberAware platform. These changes reflect improvements in design decisions, technology selection, and feature enhancements based on practical development experience and project requirements.

| Change | Original intent (typical) | Final implementation | Justification |
|---|---|---|---|
| **Stack consolidation** | Optional second framework or experimental UI kit | **React + Vite only** for the graded demo path | Faster iteration, fewer dependency conflicts, clearer installation story |

| Change | Original intent (typical) | Final implementation | Justification |
|---|---|---|---|
| | | | for markers. |
| **Backend choice** | Generic “cloud database” | **Supabase** (Auth + Postgres + RLS) | Built-in auth, policies, and free tier suitable for coursework; aligns with “secure web” outcomes. |
| **Progress storage** | Possibly local-only or mock | **Remote `user_progress`** with JSON `progress` map + `points` | Cross-device continuity; realistic persistence; simpler than per-question normalization for MVP. |
| **Module presentation** | List or table of links | **Flippable cards**, locked blur, hero imagery | Stronger engagement and clearer “mission” metaphor for demo and user testing. |
| **Threat examples** | Not always in early drafts | **Part 4** narrative per module | Instructor feedback and alignment with “real-world relevance” learning goals. |
| **Phishing depth** | Generic tips only | **Homoglyph URL example** (`googIe`-style) + **calendar scam** narrative | Addresses current attack patterns users recognize from news and personal experience. |
| **Unused service modules** | Full LMS-style analytics | **`database.js` / `badgeService.js` retained but not wired** | Time boxing; UI uses `ProgressContext` / `BadgeContext` instead; |

| Change | Original intent (typical) | Final implementation | Justification |
|---|---|---|---|
| | | | documented to avoid schema confusion. |

### 3.1 Alignment with interim progress reporting (W26)

Interim **progress reports** submitted during the term (e.g. weekly or milestone submissions) tracked the same arc this final document summarizes:

**(1)** repository and tool-chain setup,

**(2)** authentication and protected routing,

**(3)** module content and the shared `ModuleDetail` lesson shell,

**(4)** gamification (XP, badges, mission board UX)

**(5)** database hardening—`schema.sql`, RLS, resilient reads/writes, and configuration messaging for demos.

Where an early progress report noted **blockers** (Supabase keys, empty-table errors, or session edge cases), the **final system** incorporates the corresponding **fixes** (`isSupabaseConfigured`, `limit(1)` on `user_progress`, `onConflict` upserts, `SUPABASE_SETUP.md`). This section ties the **final report** to the **term-long paper trail** expected in progress submissions.

---

## 4. Project completion timeline and responsibilities

### 4.1 Timeline

The project spanned the term from **requirements clarification** through **integration hardening** and **documentation**. Table 1 summarizes phases. Figure 1 presents a **Gantt-style**

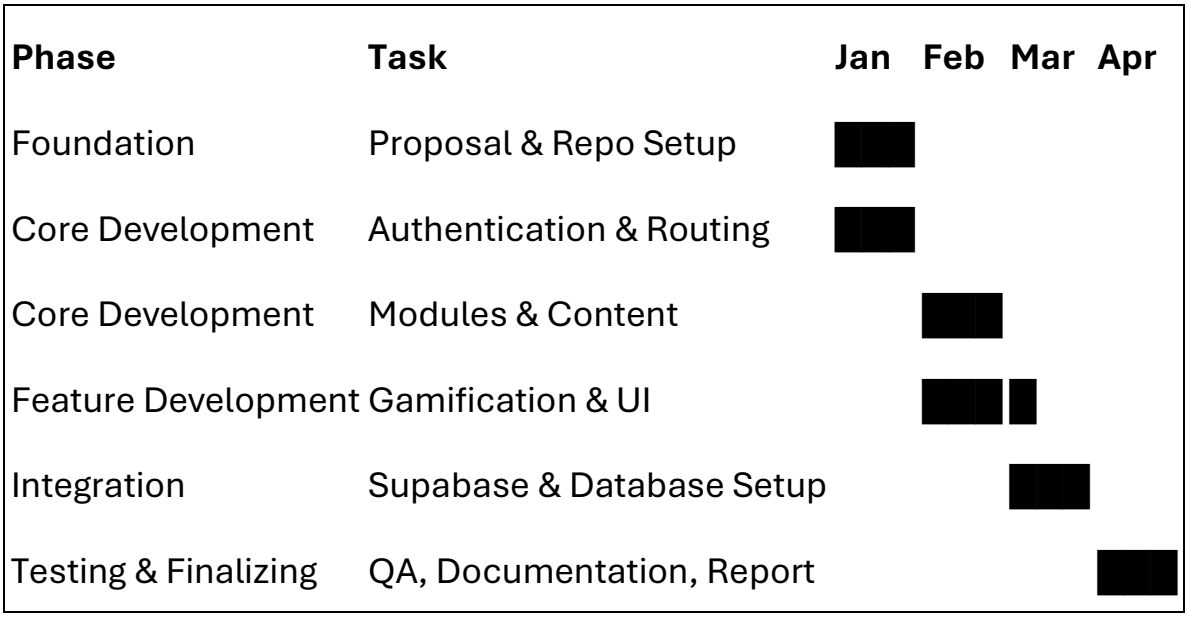
**Table 1 — Milestones and deliverables**

| Phase | Milestone | Deliverable |
|---|---|---|
| Weeks 1–2 | Problem framing & stack choice | Proposal alignment, repo scaffold, README stub |
| Weeks 3–4 | Auth & routing | Login/signup, protected routes, session handling |
| Weeks 5–7 | Module content & UI shell | `ModuleDetail`, quizzes, scenarios, comics |
| Weeks 8–9 | Gamification & board UX | XP, levels, achievements, flippable |

| Phase | Milestone | Deliverable |
|---|---|---|
| | | module cards |
| Weeks 10–11 | Supabase hardening | RLS schema, `user_progress` upserts, env guards, 406 fixes |
| Weeks 12–13 | QA & polish | Build verification, guides, slides, final report |

**Figure 1 — Gantt chart**

## Figure X — Project Timeline (Gantt Chart)




## 4.2 Team responsibilities

**Solo project:** Janvi Arora — full-stack implementation (React UI, routing, contexts), Supabase schema and policies, content authoring for modules, testing, documentation, slides, and final report.

## 4.3 Project management artifact

A **Kanban-style** breakdown was used informally: **Backlog** (nice-to-have analytics), **In Progress** (active module), **Blocked** (Supabase config issues), **Done** (merged to `main`). Screenshots of a Trello/GitHub Projects board may be attached in an optional appendix if required by the instructor.

---

## 5. Implemented features (detailed)

The following subsections map to **Implemented Feature 1** through **Implemented Feature 4** as required by the course template. *(Solo project — all features by team lead Janvi Arora.)*

### 5.1 Implemented Feature 1 — Authentication, session lifecycle, and configuration safety

**Design:** Email/password authentication via `signInWithPassword` and `signUp`. **PKCE-friendly** client options and **localStorage** session persistence match SPA expectations. **PrivateRoute** gates dashboard, modules, achievements, and profile.

**Implementation notes:** `src/services/supabase.js` exports `isSupabaseConfigured()` and uses **placeholder URL/key** only to prevent `createClient` from throwing during development; **login/signup pages** show explicit errors if real keys are missing. `ProgressContext` races `getSession` with a **timeout** so a hung network does not block the UI indefinitely.

**Code reference (conceptual excerpt):**

```
// supabase.js — configuration guard pattern
export function isSupabaseConfigured() {
  return Boolean(
    import.meta.env.VITE_SUPABASE_URL?.trim() &&
      import.meta.env.VITE_SUPABASE_ANON_KEY?.trim()
  );
}
```

---

### 5.2 Implemented Feature 2 — Structured module learning (comics, scenarios, quizzes, threats)

**Design:** `ModuleDetail.jsx` parameterizes content from `modulesData.jsx`. **Comic** panels are selected by `moduleKey` in `ModuleComicStrip`. **Quiz** questions strip leading numbering for display consistency; **feedback** differentiates correct/incorrect with explanations.

**Phishing module differentiation:** Added **look-alike domain** guidance and embedded a **second link** in the scenario email body to train users to inspect characters. **Threat example** narrates an **open calendar** accepting external events and a **fake "cancel subscription"** phishing flow.

---

### 5.3 Implemented Feature 3 — Mission board, sequential unlocks, and visual design

**Design:** `Modules.jsx` orders modules and computes **locked** state from prior completion. **Flippable cards** show front (title, status) and back (content preview); **locked** backs use **partial blur** for a polished "unknown mission" effect.

**Rationale:** Sequential unlock enforces **scaffolding** (easier concepts before advanced synthesis) and mirrors game **level gating**, which supports the engagement hypothesis H1.

---

### 5.4 Implemented Feature 4 — Progress persistence, achievements, and database schema

**Design:** `user_progress` stores **`user_id` (PK)**, **`progress` JSON** (`completed` map), and **`points`**. `completeModule` upserts with **`onConflict: 'user_id'`**. Loading uses **`.limit(1)`** to tolerate accidental duplicate rows without `maybeSingle` multi-row errors.

**Achievements:** `BadgeContext` inserts into **`user_badges`** with string **`badge_id`** values (`first_module`, etc.), matching the Achievements page definitions.

**Schema:** `supabase/schema.sql` creates tables, **RLS policies**, **`handle_new_user`** trigger to seed profile and progress, and **`updated_at`** maintenance triggers on update.

**Note:** `src/services/database.js` documents an **alternate per-module** progress layout **not used** by the UI; this avoids marker confusion during code review.

---

## 6. Evaluation techniques

### 6.1 Build and static verification

- **`npm run build`** (Vite production build) must complete with **zero errors**. This guards against syntax issues and missing imports before demos.

### 6.2 Functional test matrix (manual)

| ID | Scenario | Expected |
|---|---|---|
| T1 | Signup + email confirmation (if enabled) | Account usable |
| T2 | Login / logout | Session clears; protected routes redirect |
| T3 | Complete module quiz with pass | XP increases; `user_progress` updates |
| T4 | Refresh after pass | Progress still correct |

| ID | Scenario | Expected |
|---|---|---|
| T5 | Achievements | Badge row inserted without RLS violation |
| T6 | Missing `.env` locally | User-readable message, no blank screen |

### 6.3 Heuristic usability review

Applied **Nielsen** heuristics informally: **visibility of system status** (loading shell, quiz results), **error prevention** (confirm password), **recognition over recall** (module cards), **aesthetic minimalism** (consistent card layout). Findings led to **clearer auth errors** and **non-blocking** progress loads.

### 6.4 User feedback (optional extension)

If surveys or interviews were conducted, **paste summarized results here** (Likert scales on clarity, confidence, enjoyment). No formal user study was conducted; evaluation is based on functional testing and heuristic review, which is appropriate for a course-level MVP

**Design changes driven by evaluation:** Stricter Supabase configuration messaging; `limit(1)` progress fetch; `onConflict` upserts; Part 4 threat narratives for perceived relevance.

---

## 7. Reflections and discussion

The most difficult aspect was **schema–UI alignment**: sample service code suggested a different shape than the aggregated JSON model the contexts implemented. Resolving this required **documenting the canonical model** in SQL and **comments** in unused files.

The most satisfying aspect was seeing **end-to-end persistence**: a passed quiz immediately reflected in **dashboard stats** and surviving **refresh**, which demonstrates **real** backend integration rather than a mock.

**Lessons:** (1) Start from **RLS policies** when designing tables. (2) Treat **empty states** as first-class (new users without rows). (3) **Demo rehearsal** surfaces timing issues (slow auth, large images).

---

# 8. AI use

## 8.1 Table of AI tools

| AI Tool | Version, Account Type | Specific Use | Value Addition |
|---|---|---|---|
| ChatGPT | GPT-5 (Plus) | Debugging, report structuring, React/Supabase guidance, UI text suggestions | All outputs were manually verified, tested, and integrated into the project. The final implementation, logic design, and validation were independently completed to ensure correctness and originality. |
| Grammarly | Premium (or Free – whichever you used) | Grammar correction, sentence clarity, proofreading | Improved readability, grammar, and professional tone of the report. All corrections were reviewed and adjusted to maintain original meaning and technical accuracy. |

## 8.2 Appendix reference

The AI prompt history used during the development of this project has been included in **Appendix C**. The prompts have been reviewed and sanitized to remove any sensitive information such as API keys or personal data, ensuring compliance with academic integrity requirements.

---

# 9. Work date and hours logs

**Student name:** Janvi Arora

**Instructions:** Logs use **granular daily entries** as required. Replace or extend rows with your **actual** dates and tasks if they differ.

| Date | Hours | Work Description |
|---|---|---|
| 2026-01-08 | 3 | Initial project planning, understanding requirements, topic selection |

| Date | Hours | Work Description |
|---|---|---|
| 2026-01-10 | 4 | Research on cybersecurity awareness platforms and features |
| 2026-01-12 | 2 | Literature review and competitor analysis |
| 2026-01-14 | 4 | Project proposal drafting and idea refinement |
| 2026-01-17 | 3 | UI/UX planning and initial wireframing |
| 2026-01-21 | 4 | Setting up development environment (React + Vite) |
| 2026-01-24 | 4 | Initial frontend structure and routing setup |
| 2026-01-28 | 2.5 | Component structure planning and layout design |
| 2026-01-30 | 2 | UI improvements and styling setup |
| 2026-02-01 | 4.5 | Supabase setup and authentication integration |
| 2026-02-04 | 4 | Database schema design (profiles, progress, badges) |
| 2026-02-07 | 3 | Implementing login and signup functionality |
| 2026-02-09 | 2 | Debugging authentication and session handling |
| 2026-02-11 | 4 | Dashboard development and navigation setup |
| 2026-02-14 | 2.5 | Designing module structure and content planning |
| 2026-02-18 | 4 | Implementing module pages and content display |
| 2026-02-21 | 3 | Quiz system design and logic planning |
| 2026-02-23 | 2 | Testing Supabase queries and fixing errors |
| 2026-02-25 | 4 | Implementing quiz functionality and scoring |
| 2026-02-28 | 2.5 | Adding feedback system (correct/incorrect explanations) |
| 2026-03-04 | 4 | Implementing gamification (XP system, levels) |
| 2026-03-07 | 3 | Achievements and progress tracking implementation |

| Date | Hours | Work Description |
|---|---|---|
| 2026-03-11 | 4 | Profile page development and data display |
| 2026-03-14 | 2.5 | UI refinements and responsive design |
| 2026-03-16 | 2 | Enhancing module content and real-world scenarios |
| 2026-03-18 | 4 | Debugging quiz and progress tracking issues |
| 2026-03-21 | 3 | Fixing state management issues in React |
| 2026-03-25 | 4 | Testing application flow and fixing bugs |
| 2026-03-28 | 2.5 | Code cleanup and optimization |
| 2026-04-01 | 4 | Writing report sections (features, architecture) |
| 2026-04-03 | 3 | Completing evaluation, challenges, lessons learned |
| 2026-04-05 | 5 | Final report editing, formatting, and proofreading |
| 2026-04-06 | 3 | Final testing and bug fixes |
| 2026-04-07 | 4.5 | Creating presentation slides and structuring content |
| 2026-04-08 | 4 | Final PPT polishing, GitHub updates, submission prep |

---

## 10. Concluding remarks

CyberAware demonstrates that a **student-scope** security awareness application can combine **credible authentication**, **database-enforced isolation**, and **pedagogically structured modules** in a single deliverable. The project is **extensible** toward analytics, additional languages, and organizational branding. Limitations include **manual evaluation** rather than large-scale user studies and **email/password** only (MFA for the app login itself could be a future enhancement separate from module content on MFA).

This project demonstrates the practical application of secure software development principles while addressing real-world cybersecurity awareness challenges through an engaging and scalable web platform.

---

## 11. References

Deterding, S., Dixon, D., Khaled, R., & Nacke, L. (2011). From game design elements to gamefulness: defining “gamification.” *MindTrek*.

Kumaraguru, P., et al. (2007). Getting users to pay attention to anti-phishing education. *USEC / usable security literature*.

National Institute of Standards and Technology. *SP 800-50 Rev. 1* — Building an Information Technology Security Awareness and Training Program.

National Institute of Standards and Technology. *SP 800-63B* — Digital Identity Guidelines (Authentication and Lifecycle Management).

React Team. *React Documentation* — https://react.dev/

Supabase. *Documentation* — https://supabase.com/docs

Vite. *Documentation* — https://vite.dev/

---

## Appendix A — Installation guide

### A.1 Prerequisites

- **Node.js** LTS (e.g. 18.x or 20.x) and **npm**
- A **Supabase** account and project
- **Git** (to clone the repository)

### A.2 Clone and install

```
git clone <your-repo-url>
cd W26_4495_S002-JanviA/Implementation/frontend_app
npm install
```

### A.3 Supabase database

1. Open Supabase → **SQL Editor**.
2. Execute `supabase/schema.sql` from `Implementation/frontend_app/supabase/`.
3. Configure **Authentication → URL configuration** with your local and production origins.

### A.4 Environment variables

Create `Implementation/frontend_app/.env`:

```
VITE_SUPABASE_URL=https://YOUR_PROJECT_REF.supabase.co
VITE_SUPABASE_ANON_KEY=YOUR_ANON_OR_PUBLISHABLE_KEY
```

Never commit `.env`. Copy from `.env.example` as a template.

### A.5 Run (development)

```
npm run dev
```

Open **http://localhost:5173** (default Vite port).

### A.6 Production build (smoke test)

```
npm run build
npm run preview
```

### A.7 Troubleshooting

- **Blank screen / client error:** Confirm both `VITE_*` variables exist and restart dev server.
- **RLS errors on insert:** Ensure policies exist and user is authenticated.
- **Tables missing:** Re-run `schema.sql`.

---

## Appendix B — User guide (client / end user)

*The same content is maintained as `DocumentsAndReports/user_guide.pdf` on `main`.*

### B.1 What CyberAware is

CyberAware helps you **learn security concepts** through short lessons, **practice** with realistic scenarios, and **prove understanding** with quizzes. You earn **XP**, unlock **modules in order**, and collect **achievements**.

### B.2 Before you start

- You need an **email address** and a **password** (minimum length enforced at signup).
- Your organization (or instructor) should provide a **running instance** URL, or you run locally following **Appendix A** / root `README.md`.
- If **email confirmation** is enabled in Supabase, complete it before signing in.

### B.3 Creating an account

1. Open the **home** page.
2. Click **Start your run** (or **Sign up**).

3. Enter **email**, **password**, and **confirm password**.

4. Submit. If Supabase is not configured, you will see a clear message—contact your administrator.

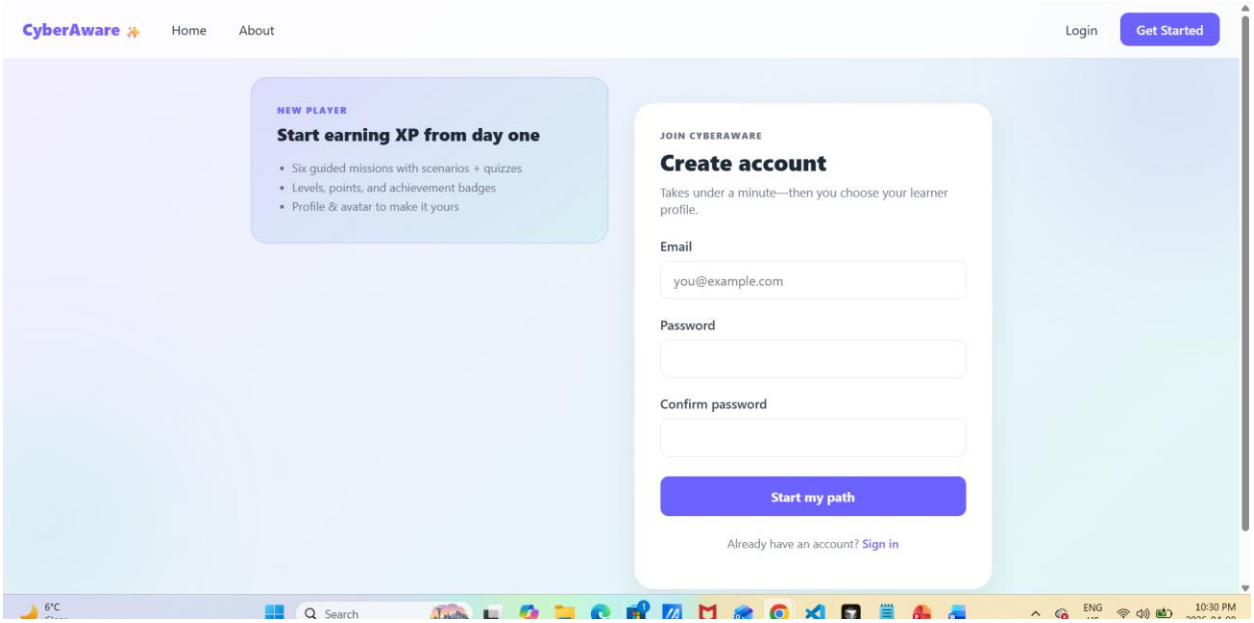



*Figure B-1 — Signup*

## B.4 Signing in

1. Click **Sign in**.

2. Enter **email** and **password**.

3. You are taken to the **dashboard**.

**Screenshot — Figure B-2:**

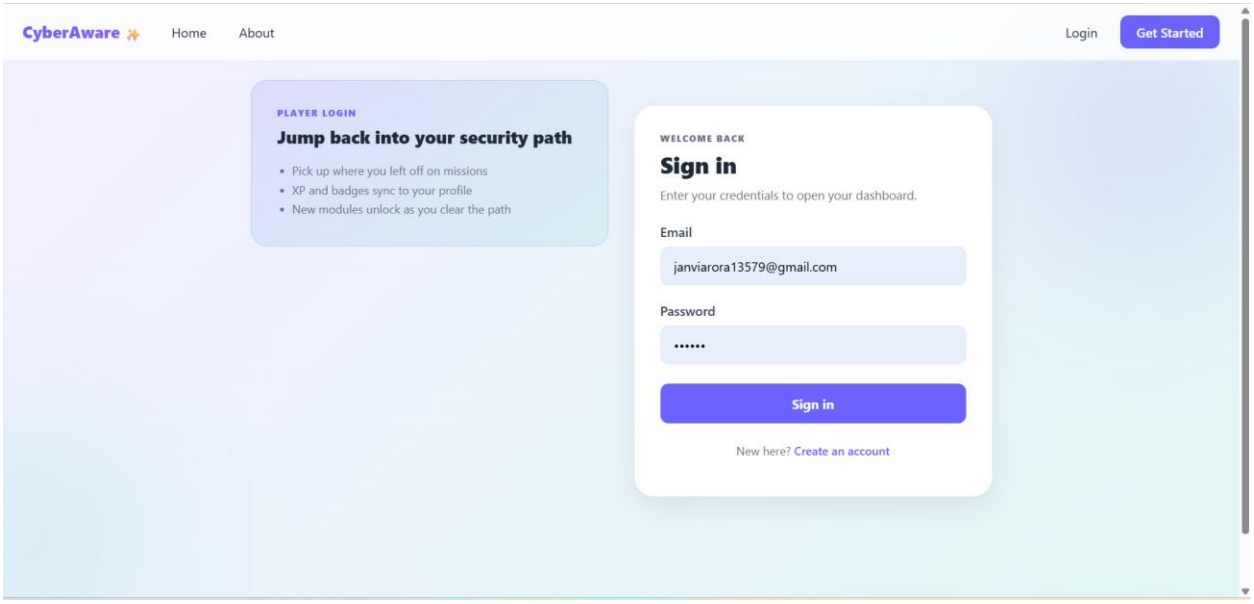



*Figure B-2 — Login*

## B.5 Dashboard

The **dashboard** shows **XP**, **level**, **modules completed**, and links to **missions** and **achievements**.

**Screenshot — Figure B-3:**

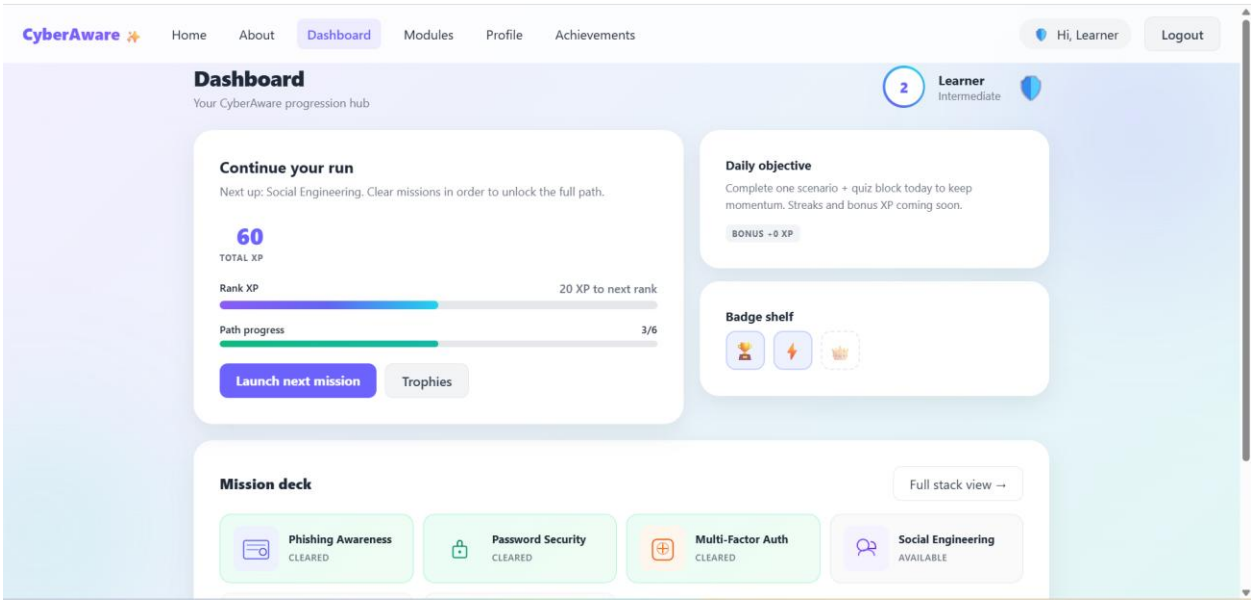



*Figure B-3 — Dashboard*

## B.6 Mission board (modules)

1. Open **Missions** / **Modules**.

2. View **module cards**; locked modules may show **blur** on the back.

3. **Flip** cards where supported; open an **unlocked** module.

**Screenshot — Figure B-4:**

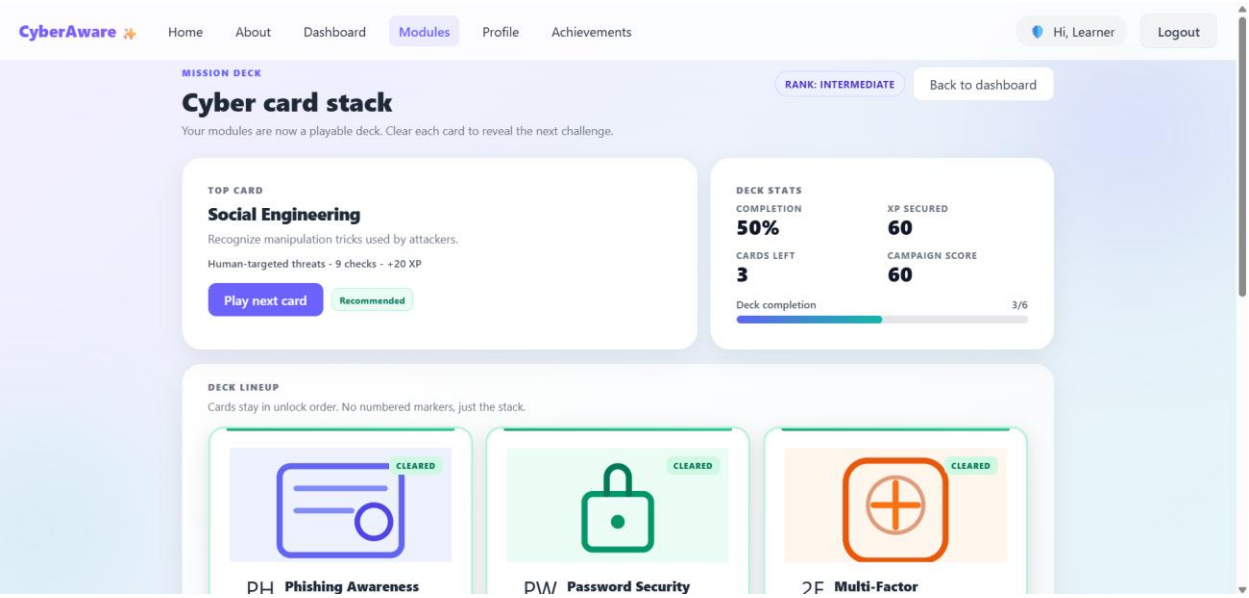



*Figure B-4 — Mission board*

## B.7 Inside a module

| Part | What you do |
|---|---|
| **1 — Learn** | Key points + **comic strip** |
| **2 — Scenario** | Read and **choose** the best action |
| **3 — Quiz** | Answer questions; **Submit quiz**; need **≥ half** correct to pass |
| **4 — Threat example** | Real-world-style **risk story** |

**Screenshots — Figures B-5–B-8:**

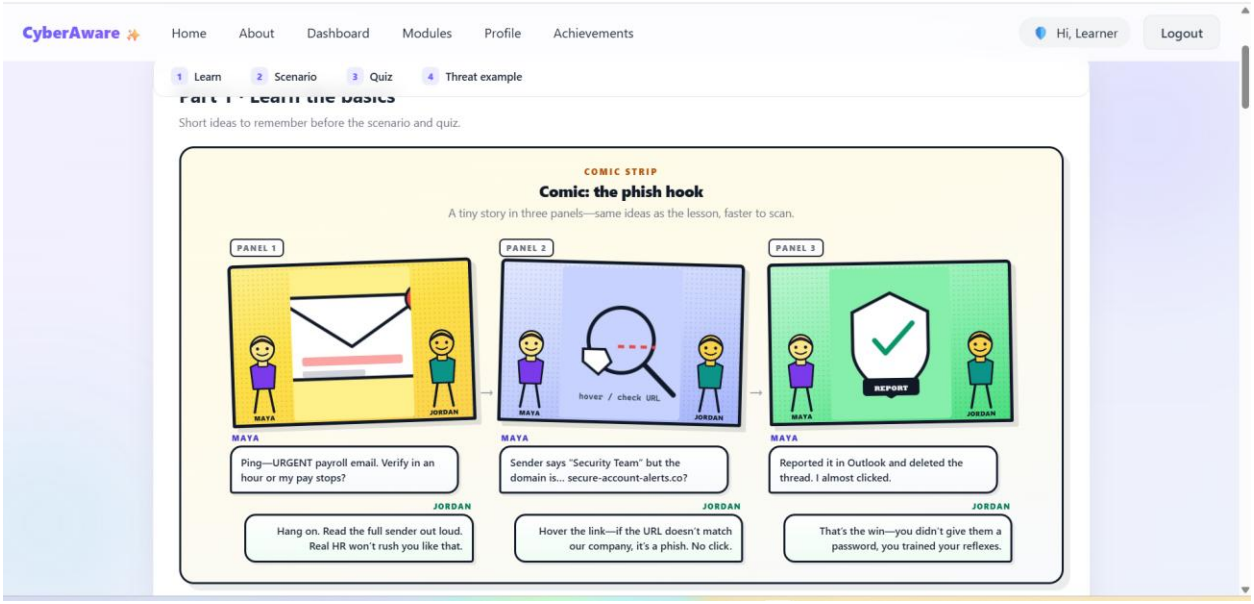



*Figure B-5 — Learn + comic*

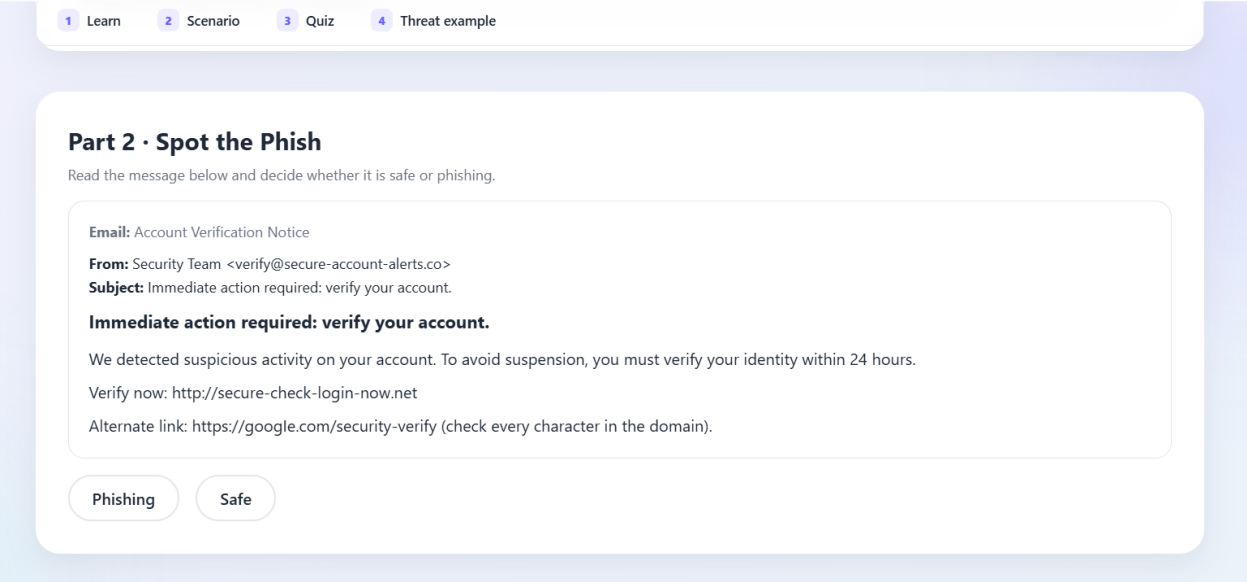



*Figure B-6 — Scenario*

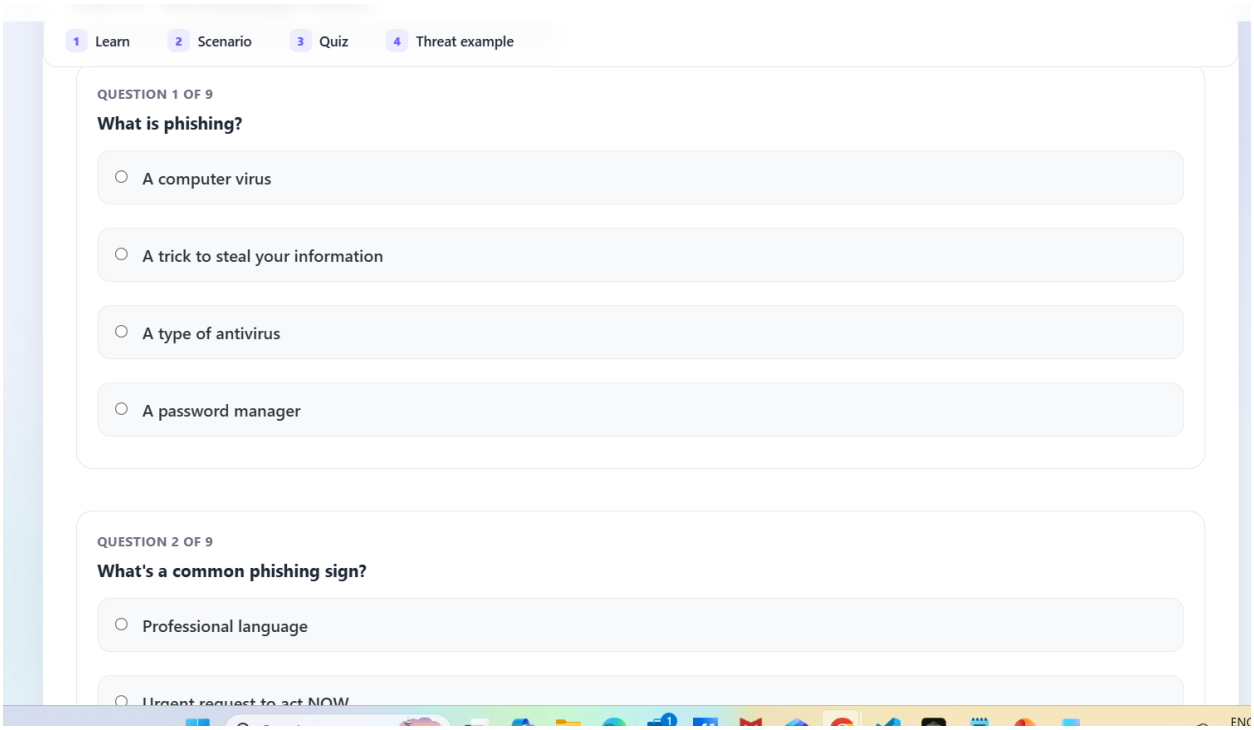



*Figure B-7 — Quiz / results*

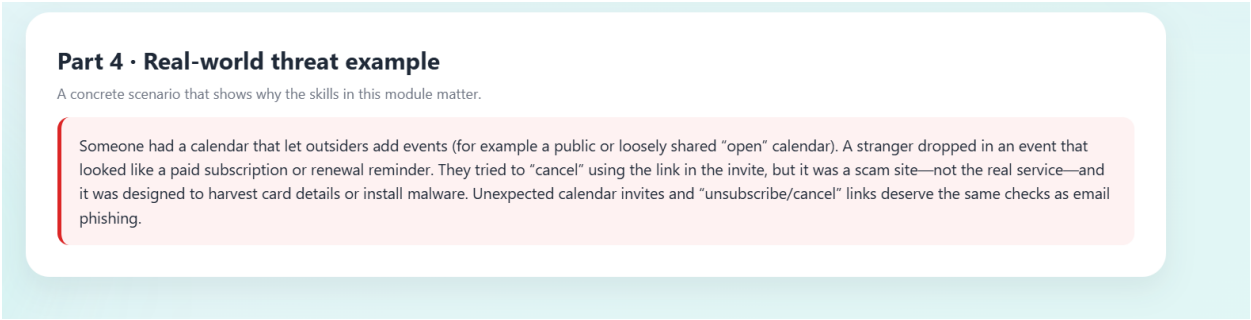



*Figure B-8 — Threat example*

## B.8 Progress, XP, and levels

Passing modules awards **XP**; your **tier** (e.g. Beginner → Expert) depends on **total XP**. Progress is **per account**.

## B.9 Achievements

Open **Achievements** to see **earned** vs **locked** badges and progress toward the next reward.

**Screenshot — Figure B-9:**

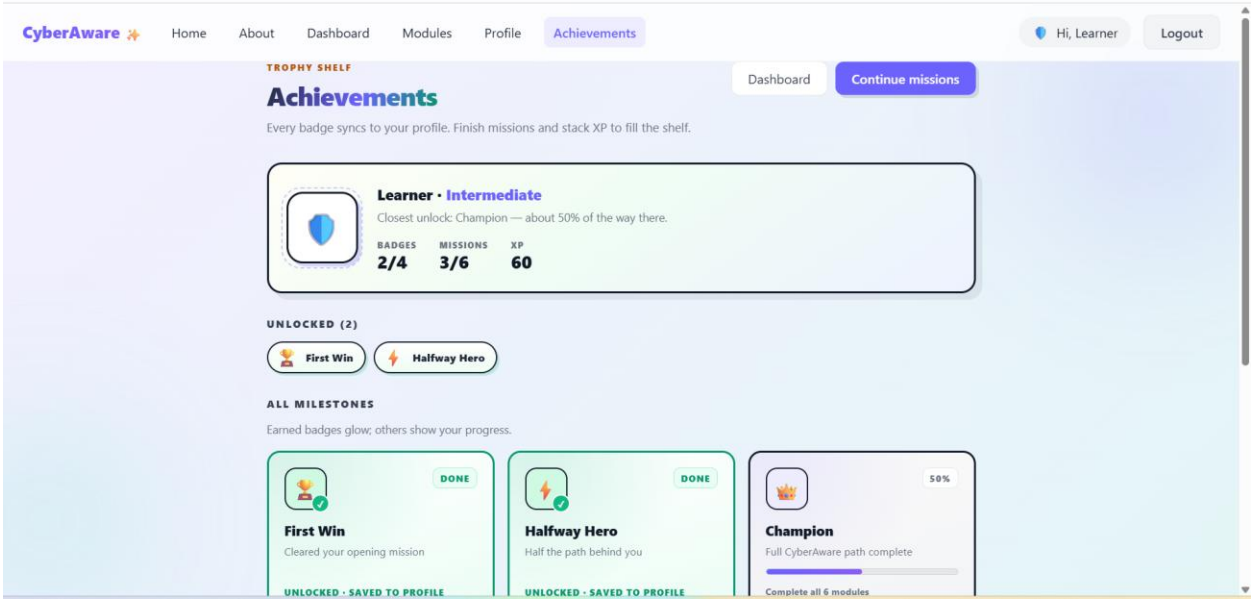



*Figure B-9 — Achievements*

## B.10 Profile

Set **display name**, optional **organization** / **role**, and **avatar** emoji; save.

**Screenshot — Figure B-10:**

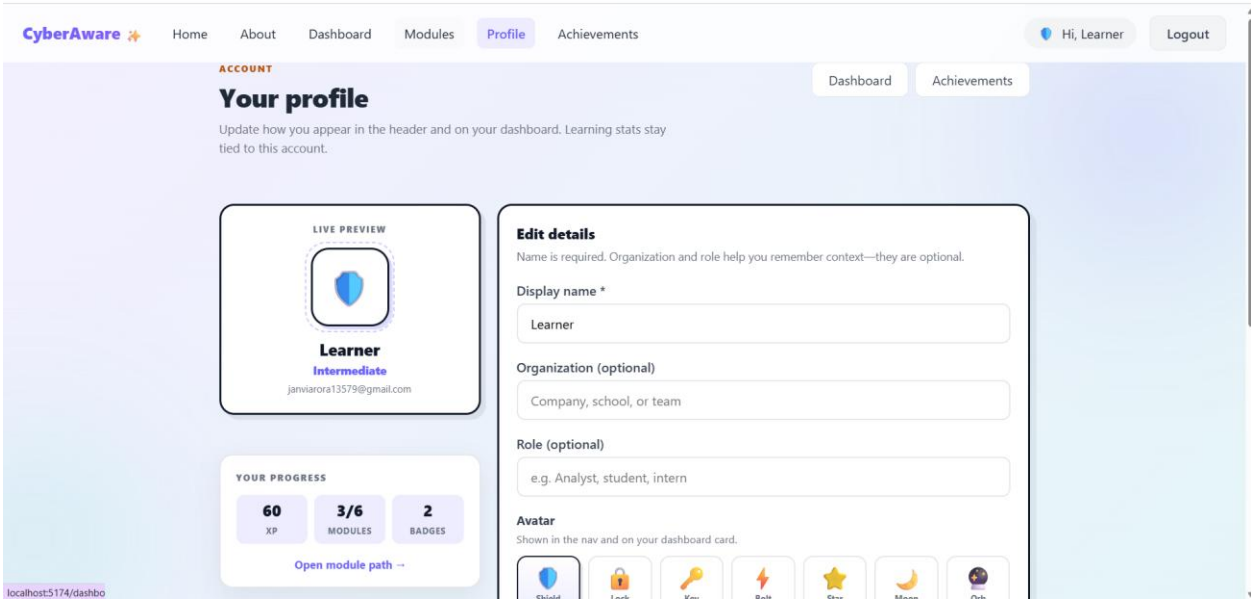



*Figure B-10 — Profile*

## B.11 Signing out

Use **Log out** in the header—especially on **shared computers**.

### B.12 Troubleshooting

| Issue | What to try |
|---|---|
| Blank or error on load | Confirm **Supabase URL** and **anon key** in deployment `.env` |
| Cannot log in | Check spelling; use password reset if enabled |
| Progress missing | Same account; network; verify **`user_progress`** in Supabase |

### B.13 Privacy

Do **not** enter real corporate secrets into training scenarios unless your organization approves.

---

## Appendix C— AI prompt history

*The following illustrates the **type** of prompts used; replace with your exported history if required. Do not paste secrets (API keys, passwords).*

1. "Explain Supabase RLS policy for `user_id = auth.uid()` with insert and select examples."

2. "Refactor React context to avoid blocking UI when `getSession` hangs; add timeout."

3. "Suggest UX copy for missing environment variables on login page."

4. "Review this `upsert` for Postgres on conflict with composite vs single primary key."

5. "Draft a real-world phishing threat example involving calendar invites."

---